Fault-Tolerant and Load-Balanced IIoT Edge–Fog Architecture

Benyapa Tawunwoot*, Proud Pitugkittikul†, Ramona Bhuthontharaj‡,
Nang Yein Mo Hsai§, Somchart Fugkeaw¶
School of Information, Computer, and Communication Technology (ICT)
Sirindhorn International Institute of Technology (SIIT), Thammasat University, Thailand
Email: *6622770509@g.siit.tu.ac.th, †6622770665@g.siit.tu.ac.th, ‡6622782140@g.siit.tu.ac.th,
§m6822041015@g.siit.tu.ac.th, ¶somchart@siit.tu.ac.th

***Abstract*—This paper proposes a fault-tolerant and load-balanced Industrial Internet of Things (IIoT) edge–fog architecture for secure, storage-efficient data processing with fault-resilient operation. The framework combines fog-scoped AES key isolation enforced by Trusted Execution Environments (TEEs), Paillier homomorphic slot aggregation, gossip-based adaptive load balancing, and sensor-side ACK-based fault detection. Fog nodes exchange status through a decentralized gossip protocol and check the most recent workload table on every incoming sensor reading. Overload is handled through KMM-provisioned enclave delegation without sensor retransmission, while ACK timeout allows sensors to redirect encrypted readings after fog silence and notify the KMM for delegated key provisioning. Vacated aggregation slots are filled with Paillier encryptions of zero, preserving aggregate correctness under mid-window delegation. The proposed scheme jointly supports fog-scoped key isolation, adaptive per-reading delegation, bounded-loss ACK-based recovery, and storage-efficient aggregation: one ciphertext per window regardless of sensor count. The 500 ms aggregation window is satisfied in non-delegation windows; delegation windows incur an additional 150 ms for TEE key provisioning.**

***Index Terms*—Industrial Internet of Things, Edge–fog computing, Fault tolerance, Load balancing, Trusted execution environment, Paillier encryption, Key management**

I. Introduction

The Industrial Internet of Things (IIoT) has become a foundational pillar of Industry 4.0, enabling continuous monitoring, predictive maintenance, and automated control across large-scale industrial environments [1]. Numerous sensors and control devices generate time-sensitive data streams that impose strict requirements on latency, reliability, and security. For example, vibration sensors on rotating machinery produce hundreds of measurements per second, while safety monitoring systems in chemical plants must detect abnormal conditions in real time to prevent hazardous incidents [2].

Centralized cloud architectures struggle to satisfy these latency requirements due to the long communication path between sensors and remote data centers. Edge and fog computing have emerged as the dominant solution, moving computation closer to data sources [3]. In an edge–fog architecture, fog nodes perform preprocessing near IIoT devices and provide intermediate aggregation and analytics before forwarding results to the cloud, significantly reducing latency and bandwidth overhead.

Despite these advantages, fog-based IIoT deployments face three critical challenges. First, heterogeneous fog nodes subject to static workload assignment create resource imbalance, where weaker nodes become overloaded while stronger nodes remain underutilized [4]. Second, fault-tolerance mechanisms in fog environments introduce extra computational and storage overhead, which affects performance in resource-constrained settings [5]. Third, many existing systems require fog nodes to decrypt sensor data before performing aggregation, violating the zero-trust assumption of modern distributed infrastructures and exposing sensitive industrial data at the fog layer [6].

Prior work addresses each challenge independently but creates new tradeoffs when combined. Global shared AES key schemes allow efficient key management but mean that a single compromised fog node exposes all sensor data across the entire deployment. Per-reading Paillier encryption [7] preserves fog-layer privacy but transmits n ciphertexts per window to the cloud, causing storage overhead that grows linearly with sensor count. Replication-based fault tolerance ensures service continuity but duplicates already-large ciphertexts, amplifying the storage problem. No existing work eliminates all three tradeoffs within a single unified architecture.

To address these limitations, this paper proposes a fault-tolerant and load-balanced IIoT edge–fog architecture. The proposed framework integrates fog-scoped AES key isolation enforced by Trusted Execution Environments (TEEs) [8], Paillier homomorphic slot aggregation [9], and gossip-based adaptive load balancing, providing end-to-end data confidentiality without exposing plaintext at any intermediate processing stage.

The primary novelty of this work is the integration of fog-scoped key isolation with a three-layer fault-handling hierarchy for fog-based IIoT. Each fog node holds only its own scoped AES key k_{fogI} sealed inside the TEE hardware, while temporary delegation is performed by the KMM provisioning the delegating node's scoped key to the receiving enclave for one aggregation window and revoking it after the window closes. The hierarchy combines: (i) fog-initiated TEE delegation when self-detected workload exceeds threshold τ, providing immediate and zero-data-loss overload handling; (ii) sensor-initiated ACK-timeout redirect using the local gossip table and a task-aware capacity scoring selector (Capaci-

tyScore, defined in Section IV-C) with KMM provisioning in approximately $150\,ms$; and (iii) gossip-based neighbor detection as a fallback for simultaneous fog-node and sensor–fog channel failure.

II. Related Work

This section reviews existing literature across three areas: (A) load balancing in fog environments, (B) fault tolerance mechanisms for distributed fog systems, and (C) privacy-preserving data processing in IIoT and fog computing.

A. Load Balancing in Fog Computing

Load balancing in fog computing is concerned with distributing incoming workloads efficiently across available nodes to prevent bottlenecks and maximise resource utilisation [4]. The challenge is compounded by the heterogeneous capacity of fog nodes which differ in CPU, memory, and network bandwidth and by the diversity of IIoT task types, which impose widely varying computational demands. Recent surveys identify three dominant strategy families: centralised scheduling, decentralised peer-to-peer coordination, and gossip-based dissemination [11], [12].

1) Optimisation-Based Approaches: Early optimisation-based methods applied metaheuristics such as Particle Swarm Optimisation and Ant Colony Optimisation to allocate workflows across fog nodes [13]. The FOCALB architecture combined fitness-based node selection with fog-aware scheduling to reduce execution time and energy consumption for scientific workflow applications [14]. While FOCALB demonstrated the value of capacity-aware routing, it targets batch workflows rather than the continuous, high-frequency sensor streams characteristic of IIoT deployments, and provides no mechanism for differentiating among task types with heterogeneous computational profiles.

Multi-criteria decision-making frameworks extended optimisation approaches by jointly evaluating CPU load, network latency, and queue depth. Ebrahim and Hafid proposed an ELECTRE-based outranking method that ranks fog nodes by pairwise concordance and discordance across multiple criteria [15]. Although this approach produces theoretically principled decisions, its $\mathcal{O}(N^2)$ pairwise comparison complexity is prohibitive under high IIoT task arrival rates. The authors explicitly deployed the method on resource-rich SDN controllers, introducing a centralised coordinator that becomes a single point of failure. A follow-up hybrid model combining FAHP and FTOPSIS improved utilisation in fog-cloud simulations [16] but similarly assumed a centralised control plane. Walia et al. proposed an adaptive multi-criteria framework for fog-cloud resource allocation that achieved load balance improvements over conventional round-robin and threshold baselines [**?**]; however, their weight assignment relied on expert judgement rather than empirical calibration against measured sensor workloads, limiting applicability to heterogeneous IIoT environments.

2) Reinforcement Learning Approaches: Reinforcement learning has been applied to fog load balancing to handle dynamic workloads without static scheduling rules. Ebrahim and Hafid proposed a Double Deep Q-Learning agent that minimises waiting time while preserving node-load privacy in multi-provider deployments [18]. More recently, the same group extended this to a fully distributed Multi-Agent Reinforcement Learning (MARL) framework in which agents operate within local collaboration regions, incorporating a gossip-based observation protocol to address the impracticality of real-time global state visibility [19]. While MARL improves scalability compared to centralised RL, these approaches require an offline training phase and may need retraining when network topology or task arrival distributions change—a significant limitation in safety-critical IIoT systems where operational continuity is mandatory. Furthermore, neither the RL nor the MARL formulations differentiate among sensor types: vibration, pressure, temperature, and flow sensors are treated as a homogeneous task class despite their substantially different computational demands.

3) Gossip-Based and Decentralised Approaches: Gossip protocols have attracted attention for fog load management because they disseminate status information among nodes through random pairwise exchanges without requiring a central coordinator. Kashani et al. surveyed gossip and epidemic-style load balancing mechanisms, noting their low per-node message overhead and natural scalability but identifying their limited integration with task-aware delegation policies as an open problem [10]. A recent systematic literature review covering 113 peer-reviewed studies from 2020 to 2024 confirmed that gossip-based coordination remains predominantly used for monitoring rather than as a complete scheduling substrate, and highlighted the lack of heterogeneous task-type differentiation as a critical gap across the field [11].

A key limitation shared by all three strategy families is their treatment of tasks as homogeneous units, typically classified only by delay sensitivity. In practice, IIoT workloads differ substantially: a vibration sensor task involving FFT-based spectral analysis is compute-intensive ($\mathcal{O}(n \log n)$), whereas a temperature monitoring task involves only threshold evaluation ($\mathcal{O}(n)$). Assigning identical routing weights to these tasks leads to suboptimal delegation decisions, particularly under heterogeneous node configurations. The proposed framework addresses this gap by introducing a CapacityScore formula with two delay-class weight configurations and a task-type compatibility term—a BW-prioritized, latency-sensitive configuration for time-critical tasks and a ϕ^2-penalised CPU-compatibility configuration for compute-heavy tasks—adapting delegation decisions to the computational and structural profile of each task class without requiring a centralised controller.

B. Fault Tolerance in Fog Computing

Fog environments are susceptible to node failures caused by hardware faults, network instability, or transient resource exhaustion. A comprehensive 2024 taxonomy of fault-tolerance

approaches in distributed and cloud computing organised existing methods into reactive, proactive, adaptive, and hybrid categories [20]. In the fog context, the dominant techniques remain replication, checkpointing, and resubmission-based scheduling.

1) Replication and Checkpointing: Replication-based methods maintain backup copies of tasks across multiple fog nodes so that computation can continue when a primary node fails. Liu et al. proposed a replica-based scheduling strategy for heterogeneous cloud-fog environments that minimises the number of replicas needed while satisfying reliability constraints [21]. Rajab and Younis introduced a dynamic fault-tolerant scheduling scheme combining checkpointing and replication that periodically stores task states, enabling recovery without full restart when failures occur [22]. Both approaches have demonstrated reliability improvements; however, they introduce additional storage overhead and computational cost proportional to the replication factor, which is problematic for resource-constrained fog nodes. Furthermore, these methods address load balancing and fault tolerance as separate concerns, leading to fragmented system designs and increased coordination overhead.

2) Metaheuristic and ML-Based Fault-Tolerant Scheduling: Hybrid metaheuristic methods have been proposed to jointly optimise workload distribution and fault-tolerant scheduling. Ghasemi proposed the MOHHO algorithm combining Modified Harris Hawks Optimisation with multi-objective service placement in fog computing, which identifies critical tasks and selectively applies replication or resubmission strategies upon node failure [23]. While these approaches improve reliability, they require offline parameter tuning and incur convergence overhead that may delay task reassignment under real-time failure conditions. Merah et al. applied a deep-learning and genetic-algorithm clustering approach to load balancing in IoT edge environments, showing adaptive traffic distribution under workload fluctuations [**?**]; however, their framework targets traffic load distribution rather than fault-tolerant task recovery, and does not address mid-window state preservation under node failure. Mohammadi et al. studied fault tolerance specifically in fog-based Social IoT environments and highlighted that combining failure detection with load redistribution in a unified lightweight mechanism remains an open challenge [25].

3) Gossip-Based Failure Detection: Gossip protocols were originally proposed for distributed database maintenance and epidemic information dissemination [26]. Their application to failure detection in fog networks follows naturally from their decentralised nature: nodes declare a peer unavailable when it fails to respond for a predefined number of consecutive gossip rounds, without requiring a centralised health monitor. Naha et al. proposed a fuzzy logic-based failure handling mechanism built on gossip-style status exchange for fog nodes [27], demonstrating that decentralised detection can adapt robustly to varying failure rates. However, this work did not integrate failure detection with task-aware load balancing in a unified scheduling framework.

The proposed framework addresses the gap between fault detection and scheduling by combining gossip-based failure detection with the same CapacityScore delegation mechanism used for overload-driven load balancing. When a node is declared unavailable after missing r consecutive gossip rounds, its pending tasks are reassigned to the highest-scoring available neighbour using the most recently maintained gossip state table, requiring no additional inter-node communication beyond what is already exchanged for load monitoring.

C. Privacy-Preserving Data Processing in IIoT and Fog Computing

Protecting sensitive IIoT sensor data during transit and processing at intermediate fog nodes is a central security requirement. Prior work has explored symmetric encryption at the sensor layer, privacy-preserving aggregation at fog nodes, and hybrid cryptographic pipelines combining these primitives.

1) Homomorphic Encryption for Fog Aggregation: Homomorphic encryption enables computation directly on ciphertexts without decryption, making it suitable for privacy-preserving aggregation at fog nodes. Zhang et al. proposed a Paillier-based privacy-preserving aggregation scheme in fog computing that demonstrated fault tolerance by handling false data injection through blinding factor adjustments [**?**]. Shang et al. combined Paillier homomorphic encryption with ECDSA signatures to build a fault-tolerant data aggregation scheme that resists collusion attacks between the fog node and the cloud [29]. A recent fog-enabled secure analytics framework (FESDAO) extended Boneh-Goh-Nissim homomorphic encryption to support not only summation but statistical operations mean, variance, and ANOVA directly on encrypted data at the fog layer, without requiring decryption before analysis [30]. These works collectively confirm the feasibility of fog-layer homomorphic aggregation; however, none addresses the dynamic workload delegation scenario in which sensors are reassigned mid-aggregation-window, which is central to the proposed framework.

The MOZAIK platform demonstrated an end-to-end privacy-preserving IoT-to-cloud architecture using both Secure Multi-Party Computation and Fully Homomorphic Encryption, showing that encrypted data pipelines can satisfy practical latency requirements [31]. However, MOZAIK targets cloud-centralised analytics and does not incorporate fog-layer load balancing or mid-window delegation semantics. The proposed framework directly addresses this gap by introducing a slot protocol that maintains aggregate correctness under dynamic sensor reassignment, using zero-fill ciphertexts to preserve IND-CPA indistinguishability of vacated slots.

2) Hybrid Cryptographic Pipelines: Practical IIoT systems increasingly combine symmetric, asymmetric, and homomorphic primitives into layered pipelines that balance security with performance. Qin et al. surveyed proxy re-encryption schemes and showed that PRE-based delegation achieves sub-15 ms overhead for typical IIoT payloads on commodity fog hardware [32], validating delegation latency as a practical concern. However, PRE requires key material to be present

at an intermediate node during re-encryption, which creates a trust boundary conflict in hostile-fog threat models.

The proposed framework replaces the PRE delegation layer with a Trusted Execution Environment (TEE) channel implemented via Intel SGX enclaves and a Key Management Module. This eliminates the need for the intermediate node to hold re-encryption key material in host memory, bounding the blast radius of fog node compromise to a single fog-scoped key group. This design follows the direction identified by recent surveys on secure fog computing, which emphasise hardware-rooted trust as the emerging approach for overcoming the limitations of software-only delegation schemes [12], [33].

III. Preliminaries

A. System Architecture

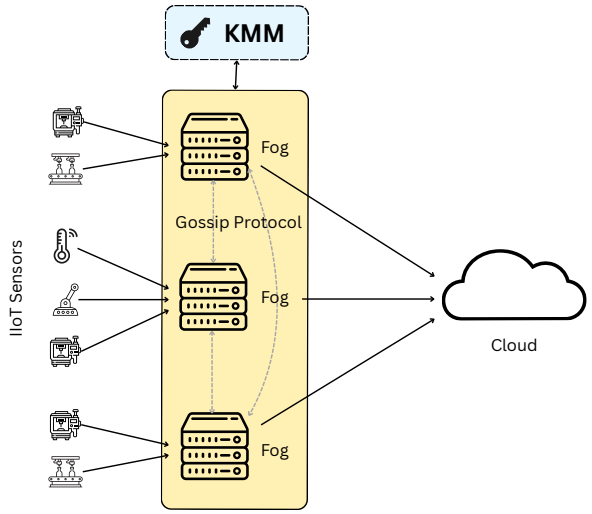

Fig. 1. Architecture of the proposed secure IIoT fog–cloud framework.

As shown in Fig. 1, the proposed system consists of five layers: the IIoT sensor layer, fog TEE enclave layer, fog Paillier aggregation layer, fog load-balancing layer, and cloud storage layer. The key management module (KMM) additionally performs a lightweight window-combine step that receives one Paillier aggregate from each fog node every $W = 500\,ms$ and combines them homomorphically.

- **IIoT Sensor Layer**: IIoT sensors collect environmental or industrial monitoring data. Each sensor sends four AES-128-GCM encrypted readings per aggregation window (one every 125 ms) using the fog-scoped key of its assigned fog node. With 100 sensors across five fog nodes (20 per node), this yields 400 readings per 500 ms window.
- **Fog TEE Enclave Layer**: Each fog node first verifies the AES-GCM authentication tag of incoming sensor readings using its provisioned fog-scoped key, rejecting readings whose tag fails verification. Accepted ciphertexts are passed to the TEE enclave, which seals its local k_{fogI}, decrypts the accepted AES readings, and encrypts scaled readings under Paillier without exposing plaintext to the host OS. sk_P and k_{store} are held exclusively in the dedicated storage-preparation enclave provisioned by the KMM at window close.
- **Fog Paillier Aggregation Layer**: Fog host memory maintains a running Paillier aggregate C_{agg,F_i} for the current $W = 500\,ms$ window using additive homomorphic operations.
- **Fog Load-Balancing Layer**: Fog nodes exchange workload status through gossip every $T_g = 500\,ms$. Delegation is checked per incoming reading using $LastKnownWorkload$ and the proposed CapacityScore selector (defined in Section IV-C), then executed through KMM-provisioned enclave processing.
- **Key Management and Window Combine**: The KMM provisions keys only after remote attestation, maintains $slot\_map$, and combines one aggregate ciphertext per fog node at window close.
- **Cloud Storage**: The cloud stores only AES-encrypted aggregates (C_{data}). Authorized clients obtain k_{store} from the KMM to decrypt the aggregate for analytics.

B. Subsystem Interaction Overview

The subsystems interact in a sequential pipeline with a conditional branch at the fog layer. Under normal operation, sensor-encrypted data flows from the IIoT Sensor Layer directly to the primary Fog Node, which verifies the AES-GCM authentication tag before the enclave converts the accepted AES ciphertext into Paillier ciphertext using that fog node's scoped key. The fog host accumulates a running Paillier aggregate during the current window. When $LastKnownWorkload(F_A) \geq \tau$ for an incoming reading, Fog A selects the receiving fog node with the proposed CapacityScore (defined in Section IV-C), zero-fills that reading's local slot, notifies the KMM, and delegates the original AES ciphertext to Fog B after the KMM provisions k_{fogA} to Enclave B. At each window close, the KMM combines one Paillier aggregate per fog node without decryption and sends the final aggregate to an enclave for storage preparation. The complete data pipeline can be summarized as

$$m_i \rightarrow C_i^{AES} \rightarrow C_i^P \rightarrow C_{agg,F_i} \rightarrow C_{agg}^{final} \rightarrow C_{data} \quad (1)$$

Sensor data is encrypted, transformed inside an attested enclave for Paillier aggregation, combined by the KMM at window close, and securely stored in the cloud.

C. Threat Model

The threat model assumes: (i) sensors are trusted but physically vulnerable to eavesdropping; each sensor holds its own copy of k_{fogI}, provisioned by the KMM at initialization, for AES-GCM encryption and for HMAC verification of gossip table summaries; (ii) the fog-node host operating system is honest-but-curious, meaning it correctly forwards messages and performs Paillier additions but may attempt to inspect plaintext, intermediate values, or enclave memory; the fog node host verifies AES-GCM tags of incoming sensor readings but does not access enclave memory; (iii) the TEE enclave on

each fog node is trusted after successful remote attestation; (iv) the KMM is trusted for fog-scoped key provisioning, slot mapping, revocation, and homomorphic window combine but never decrypts Paillier ciphertexts; (v) the cloud is honest-but-curious; and (vi) network channels between layers are subject to eavesdropping. TEE hardware prevents the fog-node host OS from accessing enclave memory. Inside each fog node's attested enclave, the sealed copy of k_{fogI} is used for AES decryption and Paillier conversion; plaintext measurements never leave the enclave. sk_P, k_{store}, and decrypted aggregates are visible only inside the dedicated storage-preparation enclave provisioned by the KMM. The model permits partial loss of the current aggregation window if a fog node fails before its aggregate is delivered to the KMM.

1) Integrity guarantee: Because the sensor and storage stages use AES-128-GCM, accepted payloads carry integrity protection under the standard nonce-uniqueness assumption. Each protected payload is produced and verified as

$$(C, T) = Enc_{GCM}(k, n, m, A) \tag{2}$$

$$m \text{ or } \perp = DecVer_{GCM}(k, n, C, A, T) \tag{3}$$

where T is an authentication tag, n is a nonce, and A is associated data. This provides payload integrity for all accepted ciphertexts; the fog node rejects any reading whose tag fails verification.

The model does not address denial-of-service attacks on the physical infrastructure.

IV. Our Proposed Scheme

The proposed scheme is organized into seven phases. The notations used in this paper are defined in Table I.

The notation in Table I is canonical and used consistently across prose, equations, and Algorithm 1.

A. System Process

The overall secure data-processing workflow of the proposed framework is illustrated in Fig. 2.

The process includes sensor-level AES-GCM encryption, fog-node tag verification, TEE-protected AES-to-Paillier conversion, Paillier slot aggregation, per-reading TEE delegation, KMM window combine, and secure AES-GCM cloud storage.

Paillier partial homomorphic encryption is selected over fully homomorphic schemes such as CKKS and BFV because the target aggregation functions—sum, mean, and weighted sum—require only additive homomorphism. This design choice eliminates noise-budget management, rotation-key generation, and polynomial-degree tuning, reducing implementation complexity while maintaining IND-CPA security under the decisional composite residuosity assumption.

*1) **Phase 1 Initialization**:* During initialization, the KMM generates fog-scoped AES keys, Paillier keys, and the storage key. After successful remote attestation, each fog enclave is provisioned only with its own sealed fog-scoped key.

$$\forall F_i: \quad k_{fogI} \leftarrow KeyGen_{AES}() \tag{4}$$

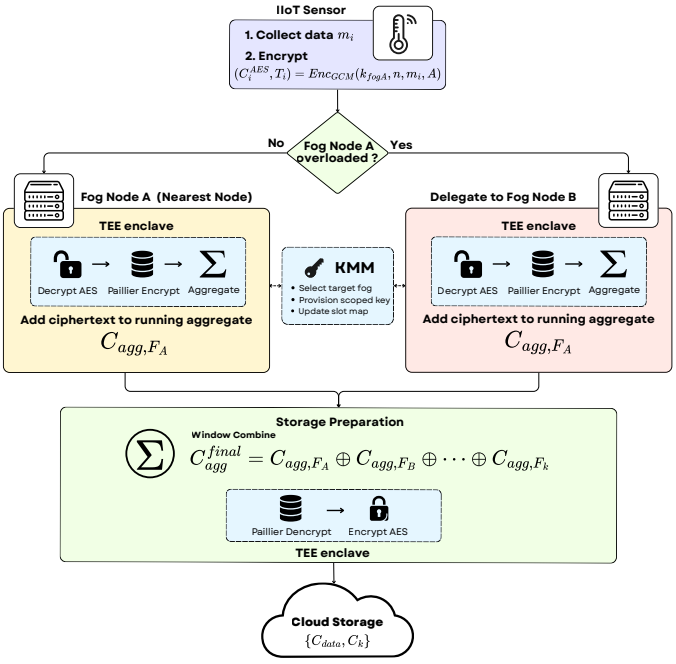

Fig. 2. Proposed Secure IIoT Data Processing Framework with TEE-Enforced Key Transformation, Paillier Slot Aggregation, and KMM Window Combine

$$(pk_P, sk_P) \leftarrow KeyGen_{Paillier}(2048) \tag{5}$$

$$k_{store} \leftarrow KeyGen_{AES}() \tag{6}$$

$$\forall F_i: \quad (ek_i, quote_i) \leftarrow TEE\_Attest(Enclave_i) \tag{7}$$

$$sealed\_kFogA = Seal(k_{fogA}, ek_A) \tag{8}$$

The KMM retains sk_P and k_{store} and seals them for its own storage-preparation enclave:

$$\begin{aligned} sealed\_sk_P &= Seal(sk_P,\, ek_{SP}), \\ sealed\_k_{store} &= Seal(k_{store},\, ek_{SP}) \end{aligned} \tag{9}$$

where ek_{SP} is the sealing key of the dedicated storage-preparation enclave. These keys are never provisioned to fog-node enclaves.

The KMM builds $slot\_map : sensor\_id \rightarrow fog\_node$, stores $fog\_node \rightarrow k_{fogI}$ for delegation provisioning, sets $W = T_g = 500\,ms$, and initializes each fog node aggregate as

$$C_{agg,F_i} = Enc_P(pk_P, 0). \tag{10}$$

*2) **Phase 2 Sensor Data Encryption**:* Each sensor encrypts its readings using AES-128-GCM under the fog-scoped key of its assigned fog node (four readings per window as stated in Section III-A). A sensor assigned to Fog A encrypts only with k_{fogA}. Upon receiving a ciphertext, the fog node verifies the AES-GCM authentication tag using its provisioned fog-scoped

TABLE I
SUMMARY OF NOTATION

Notation	Description
System Parameters	
λ	Security parameter
τ	Fog node workload threshold (scalar, set to 0.8); distinct from t_{type} (task-type classifier)
W	Aggregation window duration, set to $500\,ms$
T_g	Gossip broadcast interval, set to $500\,ms$
T_{ack}	Sensor ACK timeout for direct failure detection, set to $100\,ms$
T_{KMM}	KMM attested provisioning latency, approximately $50\,ms$
m_i	Sensor measurement at time index i
$scale$	Integer scaling factor for Paillier encoding
f	Aggregation function supported by Paillier, e.g., sum, mean, weighted sum
t_{type}	Task-type classifier: Latency-Sensitive (LS) or Heavy Processing (HP)
$slot\_map$	KMM mapping from sensor identifier to assigned fog node
$LastKnownWorkload(F_i)$	Most recent workload value for F_i from the gossip table
$LastKnownCompatLS(F_i)$	Pre-computed $compat_{LS}(F_i)$ broadcast in the gossip message; used directly in CapacityScore for LS tasks
$LastKnownCompatHP(F_i)$	Pre-computed $compat_{HP}(F_i)$ broadcast in the gossip message; used directly in CapacityScore for HP tasks
$LastKnownCapacityScore(F_i)$	Most recent full CapacityScore for F_i cached from the gossip table, assembled from $LastKnownWorkload$, $LastKnownLatency$, $LastKnownQueue$, and the appropriate pre-computed compat field
k_{gossip}	Gossip fan-out (number of neighbors contacted per round), set to 3
$miss_{gossip}(F_i)$	Number of consecutive gossip rounds in which F_i has not responded
Key Material	
k_{fogI}	Fog-scoped AES key assigned to fog node F_i
pk_P, sk_P	Paillier public and secret key pair
k_{store}	AES symmetric key for cloud-storage encryption
ek_A	TEE sealing key of enclave on Fog A
ek_B	TEE sealing key of enclave on Fog B
$quote_A$	Remote attestation quote of Fog A's enclave
$quote_B$	Remote attestation quote of Fog B's enclave
$sealed\_k$	Key material sealed under an enclave hardware key
$sealed\_kFogA$	Fog A scoped key sealed for its home enclave
$sealed\_kFogA@B$	Fog A scoped key provisioned by KMM to Enclave B for one delegation epoch
Ciphertexts	
C_i^{AES}	AES ciphertext of sensor measurement m_i
C_i^P	Paillier ciphertext of $m_i \times scale$
C_{zero}	Paillier encryption of zero for a vacated slot
C_{agg,F_i}	Running Paillier aggregate at fog node F_i
C_{agg}^{final}	KMM-combined aggregate across all fog nodes
C_{data}	$Enc_{AES}(k_{store}, \{D_{agg}, \text{metadata}\})$: single AES-GCM object stored in cloud
Cryptographic Operations	
$\text{Enc}_{AES}(k, m)$	AES encryption
$\text{Dec}_{AES}(k, C)$	AES decryption
$\text{Enc}_P(pk, m)$	Paillier encryption
$\text{Dec}_P(sk, C)$	Paillier decryption inside enclave
$\oplus$	Paillier homomorphic addition
$\text{TEE\_Attest}(E)$	Remote attestation for enclave E
$\text{Seal}(k, ek)$	Seal key material under enclave key ek
$\text{Unseal}(sealed\_k, ek)$	Recover sealed key material inside an enclave

key before passing the accepted ciphertext to the TEE enclave for processing. A single fog-node failure therefore affects 80 of 400 readings (20% of window traffic) under an even sensor distribution.

$$(C_i^{AES}, T_i) = Enc_{GCM}(k_{fogA}, n, m_i, A) \tag{11}$$

*3) **Phase 3 Per-Reading Workload Check and Domain Conversion**:* For every incoming ciphertext at Fog A, delegation is decided immediately using the latest gossip value.

$$LastKnownWorkload(F_A) \geq \tau \tag{12}$$

If the condition is false, Fog A processes the reading inside its enclave and adds the Paillier ciphertext to its running aggregate in host memory.

$$m_i = Dec_{AES}(k_{fogA}, C_i^{AES}) \tag{13}$$

$$C_i^P = Enc_P(pk_P, \lfloor m_i \cdot scale \rceil) \tag{14}$$

$$C_{agg,F_A} \leftarrow C_{agg,F_A} \oplus C_i^P \tag{15}$$

If the condition is true, Fog A zero-fills the vacated slot, notifies the KMM of the target fog node, and forwards the original AES ciphertext through the delegated processing path.

$$C_{zero} = Enc_P(pk_P, 0) \tag{16}$$

$$C_{agg,F_A} \leftarrow C_{agg,F_A} \oplus C_{zero} \tag{17}$$

*4) **Phase 4 TEE Delegation**:* When delegation is triggered, Fog A selects the target using the proposed CapacityScore (defined in Section IV-C) over available non-primary candidates from its gossip table and notifies the KMM. Candidates with $Workload(F_j) < \tau$ are preferred, with fallback to all available non-primary nodes if every candidate is saturated. Fog A does not transfer keys to Fog B, and the sensor is not contacted.

$$F_{target} = \arg\max_{F_j \in \mathcal{C}} CapacityScore(F_j,\, t_{type}) \tag{18}$$

$$Notify_{KMM}(s, F_A, F_{target}) \tag{19}$$

$$Verify(quote_B) = true \tag{20}$$

The KMM opens an attested channel to Enclave B, provisions Fog A's scoped key to Enclave B for this delegation epoch, and updates $slot\_map : sensor\ s \rightarrow Fog\ B$.

$$sealed\_kFogA@B \leftarrow Provision_{KMM \rightarrow Enclave_B}(k_{fogA}) \tag{21}$$

$$k_{fogA} \leftarrow Unseal(sealed\_kFogA@B, ek_B) \tag{22}$$

Fog B decrypts the delegated reading inside its enclave and accumulates the Paillier ciphertext in its own running aggregate. Delegated keys are revoked after the window closes, bounding the delegation exposure to one epoch.

$$m_s = Dec_{AES}(k_{fogA}, C_s^{AES}) \tag{23}$$

$$C_s^P = Enc_P(pk_P, \lfloor m_s \cdot scale \rceil) \tag{24}$$

$$C_{agg,F_B} \leftarrow C_{agg,F_B} \oplus C_s^P \tag{25}$$

$$Revoke_{KMM}(k_{fogA}, Enclave_B) \quad \text{after } W \tag{26}$$

*5) **Phase 4b ACK-Based Fault Detection with Gossip Redirect**:* Gossip-only fault detection has a detection gap of $rT_g = 3 \times 500\,ms = 1.5\,s$. During this interval, up to three aggregation windows may be affected before neighboring fog nodes identify the failed node. The proposed ACK-based mechanism reduces this gap to approximately $T_{ack} + T_{KMM} = 100\,ms + 50\,ms = 150\,ms$ by allowing sensors to detect fog-node silence directly, notify the KMM, and redirect the encrypted reading using their local gossip table and the proposed CapacityScore (Eq. (34)) LS-default selector after delegated key provisioning.

a) Operation.: Each time a sensor sends C_i^{AES} to its assigned fog node, it starts a short ACK timer. The fog node returns a lightweight acknowledgement immediately after receiving and queuing the reading. If the ACK does not arrive within T_{ack}, the sensor performs the following local redirect procedure:

- The sensor consults its local gossip status table, which is maintained using $k \times 28$ bytes of RAM.
- It selects $F_{backup} = \arg\max_{F_j} CapacityScore(F_j, \text{LS})$ while excluding the silent fog node. Because raw sensor ciphertexts carry no task-type metadata, the LS weight configuration is used by default, as sensor readings must arrive within the active 500 ms aggregation window, satisfying the latency-sensitive routing criterion.
- It sends a small redirect notification to the KMM indicating that Fog A is silent and Fog B has been selected.
- The KMM provisions $sealed\_kFogA@B$ to Enclave B through an attested channel, with expected provisioning latency of approximately $50\,ms$.
- It retransmits C_i^{AES} directly to F_{backup}.
- Enclave B decrypts with the provisioned k_{fogA}, so the blast radius is limited to Fog A's sensor group and does not expose keys for other fog nodes.

b) Sensor gossip table memory cost.: Each fog-node status entry in the sensor's local table requires 28 bytes:

TABLE II
SENSOR-SIDE GOSSIP STATUS ENTRY

Field	Size
fog_node_id	4 bytes
workload	4 bytes
latency	4 bytes
queue_length	4 bytes
timestamp	8 bytes
capacity_score_ls	4 bytes
Total per fog node	**28 bytes**

For $k = 10$ fog nodes, the table consumes $10 \times 28 = 280$ bytes. On a Cortex-M3 class IIoT sensor with 32 KB RAM, this is less than 1% of available RAM and remains negligible for the proposed deployment model. No hardware upgrade is required.

c) Sensor gossip table dissemination.: Each fog node rebroadcasts an HMAC-authenticated 280-byte gossip summary

to its downstream sensors every $T_g = 500\,\text{ms}$ ($\approx 0.56\,\text{KB/s}$, negligible on 20 Mbps or higher links).

d) Three-layer fault handling hierarchy.: The complete fault-handling system uses three layers, each covering failures that the previous layer cannot handle:

e) Interaction with the Paillier slot protocol.: When Layer 2 triggers mid-window, the slot protocol handles redirected readings similarly to a normal delegation event after the ACK timeout. If Fog A fails, its ACKs stop arriving at assigned sensors. Those sensors detect the silence within $T_{ack} = 100\,ms$ and retransmit subsequent C_i^{AES} values to Fog B. However, because Fog A is dead, it cannot zero-fill its own vacated slots in the current window. The KMM therefore handles the edge case at window close:

- The KMM detects that Fog A did not submit C_{agg,F_A} at window close.
- Fog A's contribution is marked as zero for that window.
- The KMM combines the remaining fog-node aggregates as $C_{agg}^{final} = C_{agg,F_B} \oplus C_{agg,F_C} \oplus \cdots$.
- Readings already processed by Fog A before failure are lost for that partial window if Fog A never submitted its aggregate.
- Readings after the T_{ack} redirect are captured at Fog B.

The net result is partial-window recovery. Readings processed by Fog A before failure are lost if Fog A never submits its aggregate, while readings after ACK-timeout redirect are captured at Fog B. Data loss is bounded to approximately the ACK redirect path rather than the full $1.5\,s$ gossip fallback interval.

*6) **Phase 4c Window Close and KMM Combine**:* At every $W = 500\,ms$ window close, each fog node sends one aggregate ciphertext to the KMM. The KMM performs only homomorphic addition and does not decrypt.

$$C_{agg}^{final} = C_{agg,F_1} \oplus C_{agg,F_2} \oplus \cdots \oplus C_{agg,F_k} \tag{27}$$

The KMM then sends C_{agg}^{final} to an attested enclave for storage preparation and each fog node resets its aggregate to $Enc_P(pk_P, 0)$ for the next window.

*7) **Phase 5 Secure Storage Preparation**:* The KMM forwards C_{agg}^{final} to the dedicated storage-preparation enclave, the sole holder of sk_P and k_{store}. Inside this enclave, the Paillier aggregate is decrypted and re-encrypted for compact cloud storage; both keys and the aggregate plaintext never leave enclave memory.

$$D_{agg} = Dec_P(sk_P, C_{agg}^{final}) \tag{28}$$

$$C_{data} = Enc_{AES}(k_{store}, \{D_{agg}, \text{metadata}\}) \tag{29}$$

*8) **Phase 6 Cloud Storage**:* The cloud stores only the AES-encrypted aggregate.

$$Cloud \leftarrow C_{data} \tag{30}$$

B. Trust Model: Fog-Scoped Key Isolation

The original single-global-key design creates a broad compromise domain because one broken fog enclave can expose every sensor stream. The proposed design replaces that model with fog-scoped keys, so each fog node holds only the AES key for its assigned sensor group. The KMM remains the trust anchor and provisions delegated keys only to attested enclaves for bounded delegation epochs.

The Key Management Module is the system trust anchor. It is assumed to be deployed in a hardened, high-availability environment separate from fog infrastructure. Compromise of the KMM exposes all fog-scoped keys. Compromise of a single fog node's TEE enclave, which requires physical hardware attack beyond the standard honest-but-curious fog-host threat model, exposes only that node's assigned sensor group's decryption key for the current delegation epoch. Historical data already stored as AES ciphertexts in the cloud remains protected by k_{store}, held exclusively in the KMM's storage-preparation enclave.

After each delegation window closes, the KMM revokes k_{fogA} from Enclave B. Enclave B holds the delegated key only for the duration of one $500\,ms$ window, limiting the exposure period even if Enclave B is compromised later.

The load-balancing and fault-tolerance mechanisms share the same per-reading KMM-provisioned reassignment infrastructure. Load balancing is triggered when $LastKnownWorkload(F_i) \geq \tau$ for an incoming reading. Fault tolerance is triggered either by sensor ACK timeout or by r consecutive missed gossip rounds; in both cases, the KMM provisions only the failed or overloaded fog node's scoped key to the receiving enclave. The current failure window may contain a partial aggregate if the failed node did not deliver C_{agg,F_i} to the KMM before failure.

*1) **Paillier Slot Mechanics and Storage Reduction**:* The slot protocol preserves aggregate correctness when delegation occurs mid-window. Under normal operation, Fog A accumulates all readings assigned to it as $C_{agg,F_A} = C_1^P \oplus \cdots \oplus C_n^P$. If sensor s is delegated during the same window, Fog A adds $C_{zero} = Enc_P(pk_P, 0)$ for the vacated slot while Fog B adds the delegated reading C_s^P to C_{agg,F_B}. At window close, KMM computes $C_{agg}^{final} = C_{agg,F_A} \oplus C_{agg,F_B} \oplus \cdots$, producing the same sum as if no delegation had occurred.

Paillier slot aggregation reduces the number of ciphertexts stored per window from n sensor ciphertexts to one aggregate ciphertext. For $n = 100$ sensors, this changes the per-window storage pattern from 100 individual readings to a single Paillier aggregate before compact AES storage preparation. The supported aggregation functions are sum, mean, and weighted sum; variance, standard deviation, threshold count, maximum, and minimum require multiplication or comparison and are outside the Paillier-only scope.

a) IND-CPA security of vacated slots.: Each zero-fill ciphertext $C_{zero} = \text{Enc}_P(pk_P, 0)$ is generated with a freshly sampled random coin. Under the Decisional Composite Residuosity (DCR) assumption that underlies Paillier IND-CPA security, C_{zero} is computationally indistinguishable from a

TABLE III
THREE-LAYER FAULT HANDLING HIERARCHY

Layer	Trigger	Who acts	Detection time	Data loss	Mechanism
1	Fog self-detects $Workload \geq \tau$	Fog node	Immediate per reading	Zero	KMM-provisioned TEE delegation; sensor unaware
2	No ACK within T_{ack}	Sensor and KMM	$T_{ack} + T_{KMM} \approx 150\,ms$	$\approx 150\,ms$	Sensor notifies KMM, KMM provisions k_{fogA} to Enclave B, sensor retransmits C_i^{AES}
3	$r = 3$ missed gossip rounds	Fog neighbors and KMM	$rT_g = 1.5\,s$	$\approx 1.5\,s$ fallback	Neighbor detection and KMM reassignment

TABLE IV
COMPROMISE BLAST RADIUS ACROSS KEY-PROVISIONING DESIGNS

Compromise scenario	Global AES key	All-at-init keys	Fog-scoped k_{fogI}	Reason
Fog host OS reads memory	None	None	None	TEE blocks host access to enclave memory
Fog TEE enclave physically broken	All sensors	All sensors	Only that fog node's sensor group	Fog-scoped keys bound the exposed decryption key
KMM compromised	All sensors	All sensors	All fog groups	KMM is the system trust anchor
Network eavesdropping	None	None	None	Transmissions remain AES-, Paillier-, or attested-channel protected
Fog B compromised during delegation	All sensors	All sensors	Fog A and Fog B groups only	KMM provisions only k_{fogA} to Enclave B

real reading ciphertext C_i^P. An adversary observing the host-memory aggregate C_{agg,F_A} therefore cannot determine which slots were vacated by delegation without access to sk_P, which remains sealed inside the storage-preparation enclave and is never provisioned to fog-node enclaves.

C. Load Balancing Mechanism

1) Workload Monitoring and Overload Detection: Each fog node continuously monitors its system resources to determine its processing capability. Before aggregation, each raw metric is normalized to the range $[0, 1]$ as

$$\begin{aligned} \widehat{CPU}_i &= \min\left(\frac{CPU_i^{used}}{CPU_i^{cap}}, 1\right), \\ \widehat{RAM}_i &= \min\left(\frac{RAM_i^{used}}{RAM_i^{cap}}, 1\right), \\ \widehat{BW}_i &= \min\left(\frac{BW_i^{used}}{BW_i^{cap}}, 1\right) \end{aligned} \tag{31}$$

The workload of fog node F_i is then computed as

$$Workload(F_i) = 0.5\,\widehat{CPU}_i + 0.3\,\widehat{RAM}_i + 0.2\,\widehat{BW}_i \tag{32}$$

where CPU_i^{used}, RAM_i^{used}, and BW_i^{used} denote current usage and CPU_i^{cap}, RAM_i^{cap}, and BW_i^{cap} denote node capacity for each resource. The weights 0.5, 0.3, and 0.2 reflect aggregate resource pressure across the full mix of task types: CPU dominates for compute-heavy tasks such as FFT-based spectral analysis, while bandwidth becomes the binding constraint for latency-sensitive sensor-reading tasks. This formula captures the composite overload signal used to trigger delegation; per-task routing is handled separately by the $compat$ term in the CapacityScore (Eq. (34)), which assigns an 0.80 bandwidth weight for latency-sensitive tasks and a ϕ_j^2-penalised CPU weight for heavy-processing tasks.

In this framework, $\tau = 0.8$, indicating that the node is operating at 80% or higher *normalized* composite resource utilization. This value balances responsiveness and stability: a threshold below 0.7 risks unnecessary delegation during transient load spikes, while a threshold above 0.85 leaves insufficient headroom for the additional cryptographic operations required during secure handoff.

$$Workload(F_i) \geq \tau \tag{33}$$

To maintain awareness of the system state, fog nodes exchange status information through a gossip-based communication protocol. At every gossip interval $T_g = 500\,ms$, each fog node randomly selects $k = 3$ neighboring nodes and sends a lightweight status message containing its current workload, queue length, network latency estimate, and two pre-computed compatibility scores: $compat_{LS}$ and $compat_{HP}$, evaluated locally using the node's live $\widehat{CPU}$, $\widehat{RAM}$, and $\widehat{BW}$ values before broadcast. Carrying pre-computed compat scores rather than raw resource fractions keeps the gossip payload compact while giving receiving nodes all the information needed to evaluate the full CapacityScore (Eq. (34)) without direct access to the sender's internal state.

Upon receiving a gossip message, the neighbor updates its local status table and may further propagate the information to other nodes in subsequent gossip rounds. Through this decentralized information exchange, fog nodes gradually build and maintain a $LastKnownWorkload$ table containing the most recent system state of nearby nodes. Gossip updates this table every T_g, while delegation reads the table on every incoming sensor reading; delegation therefore does not wait for either a gossip round or a window boundary.

2) Load Redistribution and Fog Node Selection: When $LastKnownWorkload(F_i) \geq \tau$ for an incoming reading, the fog node initiates a load redistribution procedure for that reading. Using the status information obtained from the gossip

protocol, the overloaded node evaluates candidate fog nodes and selects the most suitable destination for task delegation. The selection is based on a *CapacityScore* (S6), which jointly reflects a node's current load, network latency, queue occupancy, and structural compatibility with the incoming task type.

The CapacityScore of candidate fog node F_j for a task of type t_{type} is computed as

$$\begin{aligned} CapacityScore(F_j, t_{type}) = w_1(1 - Workload(F_j)) \\ + w_2(1 - \ell_j) \\ + w_3(1 - q_j) \\ + w_4 \cdot compat(F_j, t_{type}) \end{aligned} \tag{34}$$

where $Workload(F_j)$ is the normalized workload from Eq. (32). For dimensional consistency, latency and queue occupancy are normalized to $[0, 1]$ as

$$\begin{aligned} &\ell_j = \min\left(\frac{lat_j}{lat_j^{cap}}, 1\right), \\ &q_j \in [0, 1] \text{ (gossip-reported occupancy fraction)} \end{aligned} \tag{35}$$

where lat_j is the observed round-trip latency of node F_j, lat_j^{cap} is its configured latency ceiling, and $q_j \in [0, 1]$ is the queue occupancy fraction reported by the gossip protocol. The compatibility term $compat(F_j, t_{type}) \in [0, 1]$ captures the structural suitability of a node for the task type. For Latency-Sensitive (LS) tasks, which are bandwidth-dominant, it weights available bandwidth heavily:

$$\begin{aligned} compat_{LS}(F_j) = 0.10(1 - \widehat{CPU}_j) \\ + 0.10(1 - \widehat{RAM}_j) \\ + 0.80(1 - \widehat{BW}_j) \end{aligned} \tag{36}$$

For Heavy Processing (HP) tasks, which require fast CPU cores, a node-speed factor ϕ_j^2 penalises structurally slow nodes regardless of current load:

$$\begin{aligned} compat_{HP}(F_j) = 0.60\,\phi_j^2(1 - \widehat{CPU}_j) \\ + 0.30(1 - \widehat{RAM}_j) \\ + 0.10(1 - \widehat{BW}_j) \end{aligned} \tag{37}$$

The node-speed factor $\phi_j \in (0, 1]$ reflects the relative processing speed of each fog node, and execution time is modeled as $T_{exec} = base\_time/\phi_j$. The ϕ_j^2 multiplier penalises structurally slow nodes: a node with half the reference speed contributes at most one quarter of the maximum HP CPU-compatibility score, steering compute-heavy tasks away from nodes that cannot meet their execution deadlines regardless of their current load. The parameters w_1, w_2, w_3, and w_4 are dynamic weights that adapt based on the incoming task type, where

$$w_1 + w_2 + w_3 + w_4 = 1 \tag{38}$$

All score components are dimensionless in $[0, 1]$ before inversion, so the CapacityScore remains comparable across heterogeneous fog nodes. The fog node with the highest CapacityScore is selected as the delegation destination:

$$F_{target} = \arg\max_{F_j \in N_i} CapacityScore(F_j,\, t_{type}) \tag{39}$$

where N_i represents the set of neighboring fog nodes discovered through the gossip-based status exchange, excluding failed nodes and the primary node itself. The proposed method first restricts N_i to nodes with $Workload(F_j) < \tau$ and falls back to all available non-primary nodes only if no under-threshold candidate exists.

3) Task-Aware Dynamic Weight Assignment: Rather than applying fixed weights uniformly, the CapacityScore adapts based on the incoming task type. Each task submitted by an IoT device carries a *max_delay* metadata field that classifies the task into a routing-priority tier, selecting the weight configuration used in the CapacityScore formula. This field is a classification hint, not an end-to-end deadline on the fixed $W = 500\,ms$ aggregation window. The fog node uses this value to classify the task into one of two types.

TABLE V
TASK CLASSIFICATION BASED ON DELAY REQUIREMENT

Task Type	Condition	Example Applications
Latency-Sensitive	$max\_delay \leq 100\,ms$	Real-time health monitoring, industrial control systems
Heavy Processing	$max\_delay > 100\,ms$	Video frame analysis, compression tasks

Once the task type is determined, the fog node selects the corresponding weight configuration for the CapacityScore computation:

$$(w_1, w_2, w_3, w_4) = \begin{cases} (0.10,\ 0.35,\ 0.10,\ 0.45), & \text{Latency-Sensitive (LS)} \\ (0.15,\ 0.30,\ 0.10,\ 0.45), & \text{Heavy Processing (HP)} \end{cases} \tag{40}$$

In both configurations, the compatibility weight $w_4 = 0.45$ is the dominant term, ensuring that structural node suitability drives the selection decision beyond raw load conditions. For LS tasks, $w_2 = 0.35$ gives secondary priority to network latency, since these tasks are delay-critical. For HP tasks, $w_2 = 0.30$ and the ϕ_j^2 factor inside $compat_{HP}$ applies a strong structural penalty to slow nodes, steering compute-heavy tasks away from nodes that cannot meet their execution deadlines. Raw sensor ciphertexts routed during fault recovery (Layers 2 and 3) use the LS configuration by default, as explained in Section IV-C.

Because gossip refreshes $LastKnownWorkload$ every T_g while delegation is checked on every incoming ciphertext, a reading can be delegated at any time within $t \in [0, W]$ rather than waiting for the next window close.

Algorithm 1 Runtime TEE-Assisted Paillier Slot Aggregation with Load-Balanced Delegation

Require: Initialized fog nodes $\{F_1, \ldots, F_k\}$, fog-scoped keys k_{fogI}, Paillier public key pk_P, aggregation window W, workload threshold τ, and initialized aggregates $C_{agg,F_i} = Enc_P(pk_P, 0)$

Require: Latest gossip tables containing $LastKnownWorkload$, $LastKnownQueue$, $LastKnownLatency$, $LastKnownCompatLS$, and $LastKnownCompatHP$

Ensure: Updated per-fog Paillier aggregates $\{C_{agg,F_1}, \ldots, C_{agg,F_k}\}$ for the current window

for each sensor reading C_i^{AES} assigned to fog node F_A during window W **do**
  if F_A is unavailable **or** $LastKnownWorkload(F_A) \geq \tau$ **then**
    Classify $task\_type \in \{LS, HP\}$ from task metadata
    Build candidate set:

$$\mathcal{C} = \{F_j : F_j \neq F_A,\ F_j \text{ available},\ LastKnownWorkload(F_j) < \tau\}$$

    if $\mathcal{C} = \emptyset$ **then**
      $\mathcal{C} = \{F_j : F_j \neq F_A,\ F_j \text{ available}\}$ {Fallback when all candidates are saturated}
    end if
    Select target fog node:

$$F_{target} = \arg\max_{F_j \in \mathcal{C}} CapacityScore(F_j, task\_type)$$

    if F_A is still alive **then**
      $C_{zero} = Enc_P(pk_P, 0)$
      $C_{agg,F_A} \leftarrow C_{agg,F_A} \oplus C_{zero}$ {Preserve vacated slot correctness}
    end if
    Notify KMM that sensor s is delegated from F_A to F_{target}
    Forward C_i^{AES} to F_{target}
    KMM verifies $quote_{F_{target}}$
    KMM provisions $sealed\_kFogA@F_{target}$ to $Enclave_{F_{target}}$
    KMM updates $slot\_map : s \rightarrow F_{target}$
    $k_{fogA} = Unseal(sealed\_kFogA@F_{target}, ek_{F_{target}})$ inside $Enclave_{F_{target}}$
    $m_i = Dec_{AES}(k_{fogA}, C_i^{AES})$ inside $Enclave_{F_{target}}$
    $C_i^P = Enc_P(pk_P, \lfloor m_i \cdot scale \rceil)$
    $C_{agg,F_{target}} \leftarrow C_{agg,F_{target}} \oplus C_i^P$
  else
    $m_i = Dec_{AES}(k_{fogA}, C_i^{AES})$ inside $Enclave_A$
    $C_i^P = Enc_P(pk_P, \lfloor m_i \cdot scale \rceil)$
    $C_{agg,F_A} \leftarrow C_{agg,F_A} \oplus C_i^P$
  end if
end for

D. Fault Tolerance Mechanism

Fog node failures are modeled using a Poisson process [34]. The probability of p failures in interval T is

$$P(p) = \frac{(\mu T)^p e^{-\mu T}}{p!} \tag{41}$$

where μ is the average failure rate and T the observation interval. The probability of at least one failure is

$$P(p \geq 1) = 1 - P(0) = 1 - e^{-\mu T} \tag{42}$$

For gossip-based failure detection, the observation window is $T = rT_g$, where r is the consecutive-miss threshold and T_g is the gossip period. Substituting gives the probability that at least one failure *arrives* within the gossip observation window:

$$P(p \geq 1 \mid T = rT_g) = 1 - e^{-\mu r T_g} \tag{43}$$

This is a failure-arrival model, not a detector model. The deterministic worst-case detection delay of the gossip mechanism is rT_g; smaller T_g improves detection timeliness at the cost of higher gossip traffic. The arrival probability above can be used to select T_g: if a deployment requires that any failure arrives within the observation window with probability at most β, then T_g should satisfy

$$T_g \leq \frac{-\ln(1-\beta)}{\mu r} \tag{44}$$

For $\beta = 0.95$ this gives $T_g \leq \ln(20)/(\mu r)$. In the proposed system, the gossip parameters $r = 3$ and $T_g = 500\,\text{ms}$ are chosen to bound the worst-case detection delay to $rT_g = 1.5\,\text{s}$; the ACK-redirect path (Phase 4b) reduces effective recovery latency to 150 ms for the common single-node failure case.

The proposed framework uses gossip-based failure detection [26]: each fog node broadcasts its status every $T_g = 500\,ms$ to $k = 3$ randomly selected neighbors and is declared unavailable after $r = 3$ consecutive missed rounds ($rT_g = 1.5\,s$ worst-case detection). In the event of a network partition, the partitioned node initiates local failover by promoting the highest-scoring known neighbor using the most recently cached CapacityScore (LS-default configuration).

Once a failure is detected, recovery proceeds through either sensor-side ACK redirect or gossip-triggered reassignment, both using the S6 LS-default selector. In both cases, sealed key material is never exposed to the fog host OS. If the failed node did not deliver C_{agg,F_i} before failure, that window is treated as a partial aggregate; subsequent windows are handled by the promoted target node.

V. Performance Evaluation

A. Experimental Setup

All experiments are implemented in Python 3.13 using the `phe` 1.5.0 Paillier library with 2048-bit RSA moduli and a fixed-point scaling factor of $scale = 1000$. The simulation environment consists of five heterogeneous fog nodes F_1–F_5 whose hardware profiles are summarized in Table VI. The

Algorithm 2 Fault Recovery, Window Combine, and Secure Storage

Require: Gossip table, T_g, r, T_{ack}, τ, aggregates $\{C_{agg,F_i}\}$, sk_P, k_{store}

Ensure: Cloud stores C_{data}

Each fog node computes $compat_{LS}$ and $compat_{HP}$ locally from live $\widehat{CPU}$, $\widehat{RAM}$, $\widehat{BW}$

Each fog node gossips $\{workload, queue, latency, \phi, compat_{LS}, compat_{HP}, timestamp\}$ every T_g

Update $LastKnownWorkload$, $LastKnownQueue$, $LastKnownLatency$, $LastKnownCompatLS$, and $LastKnownCompatHP$

if $miss_{gossip}(F_A) \geq r$ **then**

$$\mathcal{C} \leftarrow \{F_j : F_j \neq F_A,\ F_j \text{ available},\ LastKnownWorkload(F_j) < \tau\}$$

  if $\mathcal{C} = \emptyset$ **then**

    $\mathcal{C} \leftarrow \{F_j : F_j \neq F_A,\ F_j \text{ available}\}$

  end if

  $F_{target} \leftarrow \arg\max_{F_j \in \mathcal{C}} LastKnownCapacityScore_{LS}(F_j)$

  Mark F_A unavailable and redirect its remaining sensors to F_{target}

  KMM provisions $sealed\_kFogA@F_{target}$ to $Enclave_{F_{target}}$

  KMM updates affected $slot\_map$ entries

end if

if sensor s misses ACK from F_A within T_{ack} **then**

  $F_{backup} \leftarrow \arg\max_{F_j \neq F_A} LocalGossipCapScore(F_j)$

  Sensor notifies KMM and retransmits C_i^{AES} to F_{backup}

  KMM provisions $sealed\_kFogA@F_{backup}$ to $Enclave_{F_{backup}}$

  $Enclave_{F_{backup}}$ decrypts C_i^{AES}, converts it to C_i^P, and updates $C_{agg,F_{backup}}$

end if

At window close, each available fog node sends C_{agg,F_i} to KMM

Missing fog-node aggregates are treated as zero for the current window

KMM computes $C_{agg}^{final} \leftarrow C_{agg,F_1} \oplus \cdots \oplus C_{agg,F_k}$

Reset each $C_{agg,F_i} \leftarrow Enc_P(pk_P, 0)$

Revoke delegated fog-scoped keys from receiving enclaves

$D_{agg} \leftarrow Dec_P(sk_P, C_{agg}^{final})$ inside storage-preparation enclave

$C_{data} \leftarrow Enc_{AES}(k_{store}, \{D_{agg}, metadata\})$

Store C_{data} in cloud

nodes range from a weak single-core node (F_3: 1 CPU core, 2 GB RAM, 20 Mbps) to strong quad-core nodes (F_1, F_5: 4 CPU cores, 8 GB RAM, 100 Mbps), each serving 20 sensors for a total of 100 sensors. The overload threshold is $\tau = 0.8$, the aggregation window is $W = T_g = 500\,\text{ms}$, the sensor ACK timeout is $T_{ack} = 100\,\text{ms}$, and the KMM provisioning latency is $T_{KMM} = 50\,\text{ms}$.

TABLE VI
FOG NODE HARDWARE PROFILES USED IN SIMULATION

Node	Class	CPU Cores	RAM (GB)	BW (Mbps)	Sensors	ϕ_j
F_1	Strong	4	8	100	20	1.00
F_2	Medium	2	4	50	20	0.50
F_3	Weak	1	2	20	20	0.25
F_4	Medium-Fast	2	4	80	20	0.50
F_5	Strong	4	8	100	20	1.00

ϕ_j = CPU cores$_j$ / $\max_j$(CPU cores); normalized to the strongest node (4 cores).

The TEE layer is emulated using OP-TEE running on QEMU (ARM TrustZone emulation). Enclave operations—AES-GCM decryption, Paillier encryption, and key unsealing—are modeled as OP-TEE Trusted Applications executing within the secure world. The `sgx_enclave_process` function in the prototype implements these operations using the OP-TEE QEMU environment; no actual SGX hardware boundary, SGX sealing-key derivation, or SGX remote attestation exchange is present in the simulation. Measured latencies in E2 are host-reference OP-TEE QEMU values and should not be interpreted as physical hardware timings. The architectural design is expressed using generic TEE terminology (sealing, remote attestation, enclave quotes); Intel SGX is used as the reference implementation because Intel SGX and ARM TrustZone/OP-TEE provide equivalent security primitives under the same TEE trust model. Validation on physical SGX or TrustZone hardware is reserved for future work.

Experiments E1–E2, E3a–E3b, and E4 evaluate the prototype implementation using a calibrated 2048-bit host-reference Paillier benchmark (`host_reference_2048_no_ta` mode). E5 is a simulation study averaged over 20 seeds $\{42, \ldots, 61\}$ with 95% CI. E6, E7, and E8 are analytical models. All results are organized into three thematic groups below.

B. Storage and Latency Efficiency

Storage reduction (E1). For sensor counts $n \in \{10, 50, 100, 200, 500, 1000\}$, cloud storage is compared under four schemes: plaintext ($n \times 8$ bytes), AES-GCM per-reading ($n \times 36$ bytes), Paillier without aggregation ($n \times 512$ bytes), and the proposed method (one fixed-size C_{data} per window). As shown in Fig. 3, the proposed method stores exactly one ciphertext per window regardless of n, achieving $100\times$ byte reduction over AES-GCM per-reading at $n = 100$ and $1000\times$ at $n = 1000$. Paillier slot aggregation thus reduces cloud storage from $O(n)$ to $O(1)$.

Aggregation latency (E2). Using calibrated 2048-bit Paillier timings, the proposed pipeline requires 444.128 ms in

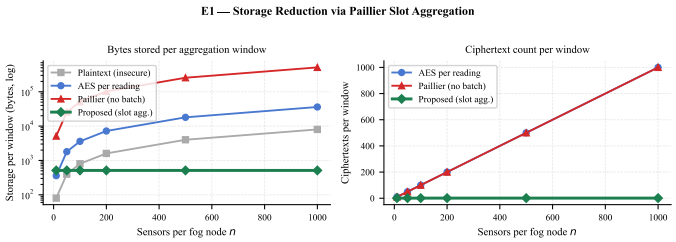

Fig. 3. Storage comparison across schemes vs sensor count n. Upper panel: total bytes; lower panel: ciphertext count. The proposed method stores one ciphertext per window for all n.

total: 227.537 ms for fog TA conversion, 1.474 ms for host aggregation and KMM window combine, and 215.117 ms for storage TA preparation (Fig. 4). The total is within the 500 ms aggregation window; fog conversion and storage preparation dominate, while the KMM combine step contributes only ≈ 6 ms for the five-node deployment evaluated here (scaling linearly with k; see E4). These timings are host-reference values from the OP-TEE QEMU simulation described in Section V-A and should not be interpreted as physical hardware timings. The discrepancy between this 444.128 ms measured figure and the 399 ms hardware-target estimate below reflects the absence of AES-NI acceleration and a hardware Paillier co-processor in the QEMU environment.

Claim (Latency Budget): Under the hardware-target analytical model, the proposed pipeline completes within $W = 500$ ms in non-delegation windows (399 ms). When delegation is triggered, the window budget is exceeded by exactly $T_{KMM} \approx 150$ ms for TEE key provisioning—an overhead that is bounded, one-time per delegation event, and predictable at scheduling time. In the 20-window evaluation schedule, the 12 non-delegation windows (W1–5, W14–17, and the normal-load portions of W11–13) complete at 399 ms. The 8 delegation-burst windows (W6–10, W18–20) each incur this fixed 150 ms overhead; deployments serving burst phases should provision a 650 ms extended window or absorb the overhead through capacity headroom. Fig. 5 shows the per-window breakdown across all 20 windows.

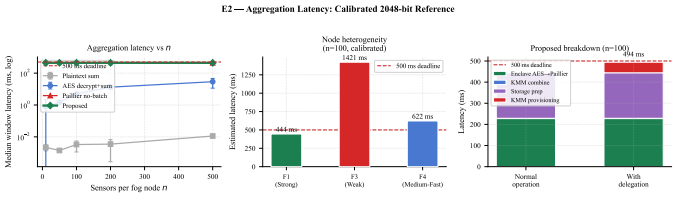

Fig. 4. Aggregation latency vs sensor count n with calibrated 2048-bit host-reference Paillier timings. The dashed line marks 500 ms. The right panel shows the component breakdown.

Table VII places the proposed pipeline within the landscape of fog architectures by comparing per-window cloud storage and latency sources across four representative methods under identical hardware-target assumptions ($n = 100$ sensors). All latency estimates are analytical; no physical measurements were conducted.

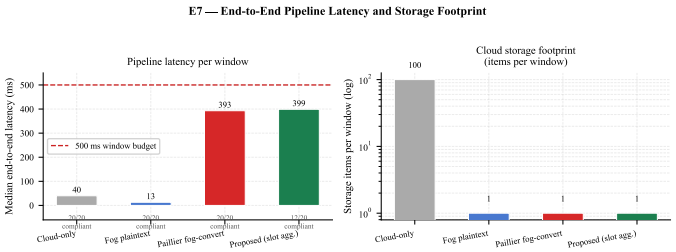

Fig. 5. Analytical pipeline latency (left) and per-window storage items (right) across 20 windows under hardware-target constants. The dashed line marks 500 ms; delegation windows incur a bounded $T_{KMM} = 150$ ms overhead above the non-delegation baseline of 399 ms.

TABLE VII
END-TO-END PIPELINE LATENCY AND STORAGE: BASELINE COMPARISON ($n = 100$, HARDWARE-TARGET ANALYTICAL MODEL)

Method	Storage/window	Latency source
Cloud-only [?]	n raw readings ($n \times 8$ B)	Sensor AES (optional) + full network path to cloud
Fog plaintext [?]	1 plaintext sum	Fog compute only ($\approx$1 ms); no cryptographic overhead
P2-SWAN style [?], [9]	n Paillier ciphertexts ($n \times 512$ B)	$n \times T_{\text{enc}}$ Paillier; no slot compression
Proposed	1 Paillier ciphertext (512 B)	AES decrypt + Paillier convert + slot aggregate

Cloud-only incurs the highest network overhead and foregoes all fog-layer latency reduction. Fog-plaintext achieves the lowest latency but exposes raw readings at the fog node, violating the zero-trust assumption. P2-SWAN-style per-reading Paillier encryption preserves privacy but transmits $O(n)$ ciphertexts per window; the proposed Paillier slot aggregation reduces this to $O(1)$ without sacrificing encrypted computation.

Slot protocol correctness (E3a–E3b). The slot protocol was verified over 100 independent trials under both single-source delegation ($F_1 \rightarrow F_4$, $k \in \{1, 5, 10, 20, 50\}$) and simultaneous dual-source delegation ($F_1, F_2 \rightarrow F_4$, each source delegating 5 readings from a pool of 30, backup node holding 30 own readings; 90 sensors total). All 200 trials produced zero scaled-integer arithmetic error. The residual quantization deviation of $\approx$0.045 (median, single-source) and $\approx$0.041 (dual-source) is fixed-point noise inherent to $scale = 1000$ Paillier encoding, independent of k and delegation source count, confirming that zero-fill correctness extends to concurrent multi-source delegation.

KMM combine overhead (E4). The KMM window-combine latency was measured at $k \in \{1, 2, 5, 10, 20, 50, 100\}$ fog aggregates. Combining k ciphertexts requires $k-1$ sequential Paillier homomorphic additions; at the calibrated single-addition cost of $\approx$ 1.474 ms, total combine latency scales as $\approx 1.474(k-1)$ ms. As shown in Fig. 8, the measured latency follows this linear trend: $\approx$6 ms at $k = 5$, $\approx$28 ms at $k = 20$, and $\approx$150 ms at $k = 100$. All tested values remain well within the 500 ms aggregation window, confirming that

the KMM combine step does not become a bottleneck across the evaluated deployment scales. For the five-node deployment evaluated in E2, the combine contributes only ≈ 6 ms of the total 444.128 ms pipeline latency.

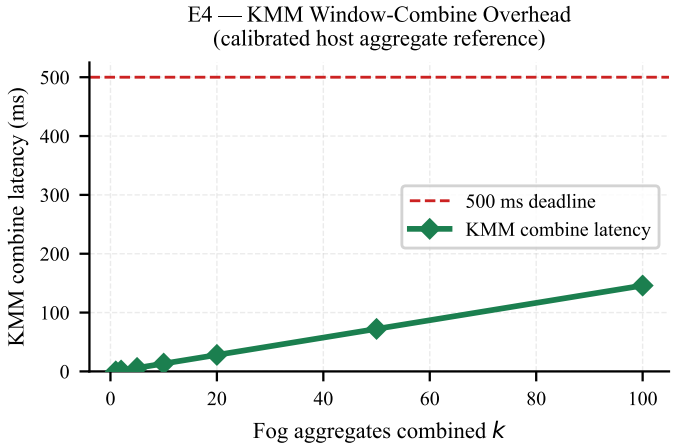

Fig. 6. E4 KMM window-combine latency vs number of fog aggregates $k \in \{1, \ldots, 100\}$. Latency scales linearly as $\approx 1.474(k-1)$ ms, reaching $\approx$150 ms at $k = 100$; all values remain within the 500 ms aggregation window.

C. Load Balancing Performance

Setup. Five fog nodes run 20 aggregation windows of 500 ms across five phases: normal operation (W1–5), F_1 burst (W6–10), F_5 failure (W11–13), recovery (W14–17), and F_4 burst (W18–20). Tasks split into latency-sensitive (LS, 100 ms deadline) and heavy-processing (HP, 500 ms deadline) classes. Six strategies are compared: S1 (random), S2 (round-robin), S3 (minimum workload), S4 (fixed CapScore weights), S5 (task-adaptive weights, no compatibility term), and S6 (proposed CapacityScore with compatibility term). Results are averaged over 20 seeds with 95% CI. This experiment assumes instantaneous workload visibility to isolate strategy performance; gossip-induced staleness ($T_g = 500$ ms) is a separate concern evaluated in the fault-tolerance analysis (E6).

Results. All six strategies complete 100% of tasks; the discriminating metrics are deadline satisfaction and routing quality (Fig. 9). S6 achieves the highest deadline satisfaction at 78.77% ($\pm$0.88%), versus 78.35% for S5, 77.28% for S3, 77.01% for S4, and 63.1–63.3% for S1–S2. S6 also has the lowest mean waiting time at 101.11 ms ($\pm$3.94 ms), compared with 103–104 ms for S3–S5 and 790–854 ms for S1–S2. The 0.42 percentage-point margin between S6 and S5 lies within the confidence intervals; the advantage of the compatibility term is more conclusively demonstrated by the task-type results. S6 has the lowest bandwidth-utilization standard deviation ($\sigma_{BW} = 0.225 \pm 0.005$ vs 0.239 for S5, 0.363 for S1–S2), accepting slightly less even scalar workload balance to reduce deadline misses.

The task-type breakdown (Fig. 10) explains the improvement. HP tasks miss deadlines at 15.18% under S6, compared with 17.70–18.67% for S3–S5 and 34–35% for S1–S2. S6 assigns only 1.97% of HP tasks to F_3 (the slowest node, which executes HP tasks in 600 ms, exceeding the deadline), versus 3.32–3.77% for S3–S5 and 13.8–14.0% for S1–S2. Phase-level results show S6 leads during normal operation (92.67%), recovery (93.83%), and F_4 burst (84.51%). During F_5 failure, S5 performs best (48.22%) while S6 drops to 37.93%: the compatibility-heavy weighting ($w_4 = 0.45$) narrows candidate selection further when the pool shrinks due to failure. Deployments anticipating simultaneous failure and high arrival rates may reduce w_4 during degraded-mode operation.

Key takeaway. CapacityScore improves deadline satisfaction by routing HP tasks away from structurally incompatible nodes and reducing bandwidth variance — not by completing more tasks, but by meeting deadlines under heterogeneous capacity and phased stress.

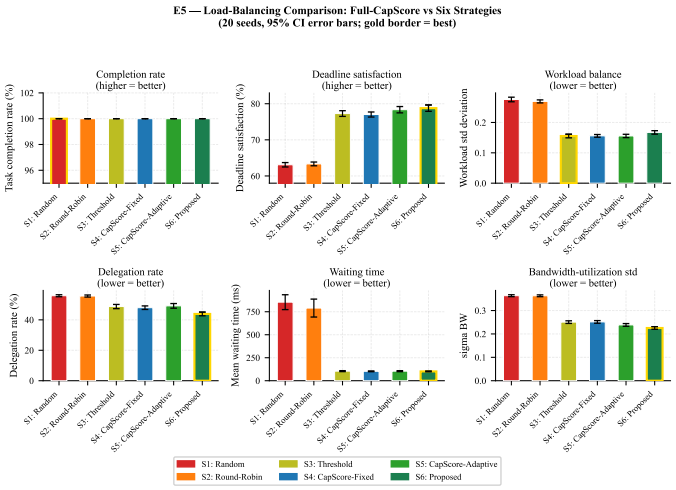

Fig. 7. Six-strategy load-balancing comparison: completion rate, deadline satisfaction, delegation rate, mean waiting time, and bandwidth-utilization standard deviation over 20 seeds with 95% CI.

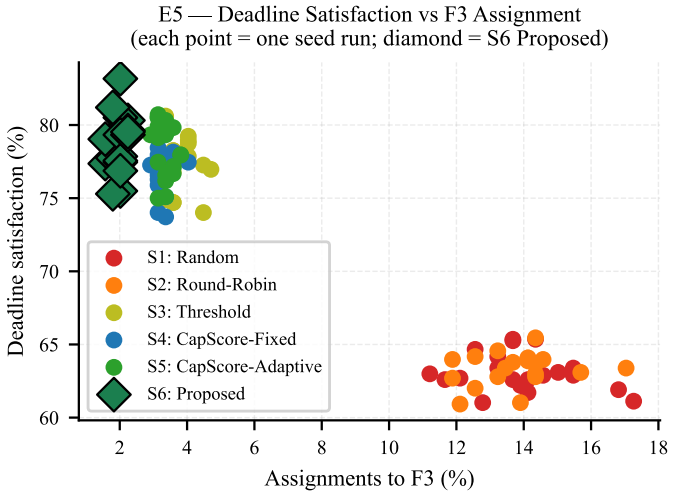

Fig. 8. Deadline satisfaction vs HP task assignment rate to F_3 across S1–S6 over 20 seeds. S6 reduces HP tasks routed to the slowest node, improving overall deadline satisfaction.

D. Fault Tolerance and Security

Fault recovery (E6). An analytical reading-loss model is evaluated over 30 seeds under three failure timings: failure at $t = 0$ ms, 250 ms, and 450 ms within a 2000 ms observation window (400 total readings). Five baselines are compared (Table VIII): B1 gossip-only, B2 replication [35], B3 checkpoint sub-mechanism (derived from the RPC policy in [36]), B4 multilayer detection [37], and B5 fog-clustering [25]. The proposed ACK+KMM method uses $T_{ack} = 100$ ms plus $T_{KMM} = 50$ ms provisioning, yielding 150 ms recovery latency.

TABLE VIII
FAULT-TOLERANCE BASELINES

	Simulator abstraction
B1	Gossip-only: failure detected after $rT_g = 1500$ ms; readings in outage window lost.
B2	Replication [35]: each reading sent to primary and backup; zero loss, 2× message cost.
B3	Checkpoint [36] (checkpointing sub-mechanism of RPC isolated for comparison; 500 ms interval): readings after latest checkpoint lost until restore.
B4	Multilayer detection [37]: sensor/fog/KMM detectors; earliest layer result used.
B5	Clustering [25]: coordinator detects failure and reroutes subsequent readings.

In the worst case (failure at $t = 0$), the proposed method loses ≈7.5% of readings (30 of 400 during the 150 ms recovery window), versus 75% for B1 gossip, ≈38% for B5 clustering, and 25% each for B3 and B4. B2 replication achieves zero loss at 2× message overhead; the proposed method incurs only 1.055×. Fig. 11 shows loss rates and recovery latency; Fig. 12 shows the loss–overhead tradeoff, where the proposed method is nearest to the origin among single-mechanism approaches.

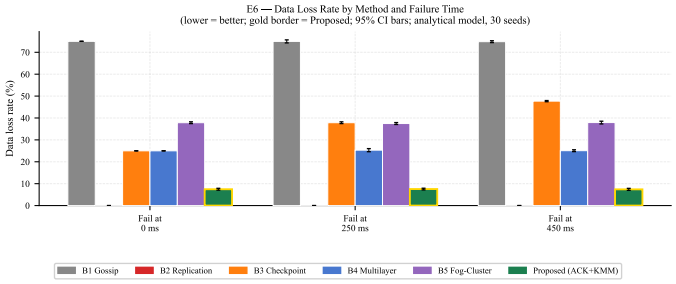

Fig. 9. Data loss rate (upper) and recovery latency (lower) across six methods and three failure timings, averaged over 30 seeds with 95% CI.

Key exposure blast radius (E8). Four compromise scenarios are evaluated analytically for 100 sensors across five fog nodes (20 per node). Under single-node enclave compromise (F_1), fog-scoped keys expose only 20 sensors versus all 100 under a global key (80% reduction). Under backup-node compromise during active delegation (F_4 holding provisioned k_{fogA}), 40 sensors are exposed versus 100 (60% reduction). KMM compromise exposes all 100 sensors under both designs

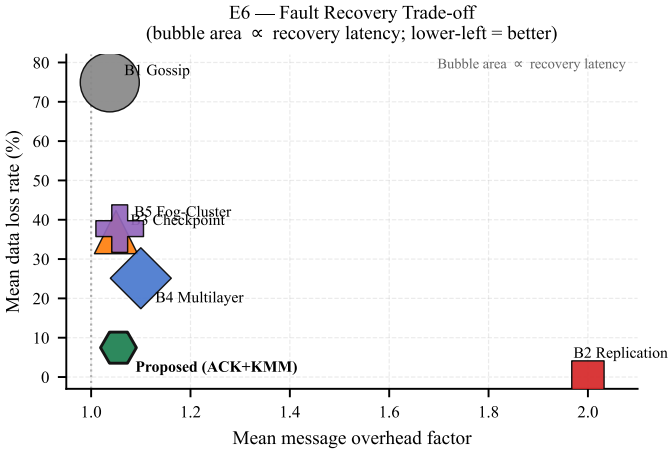

Fig. 10. Data loss vs message overhead tradeoff. The proposed ACK+KMM method (green) is closest to the origin among single-mechanism approaches.

since the KMM is the common trust anchor. Host OS memory access exposes zero sensors under both designs due to TEE enclave isolation. Fig. 13 summarizes all scenarios.

Key takeaway. Fog-scoped key isolation reduces the blast radius by 60–80% in single-node compromise scenarios. The benefit does not extend to KMM compromise, the identified critical single point of failure.

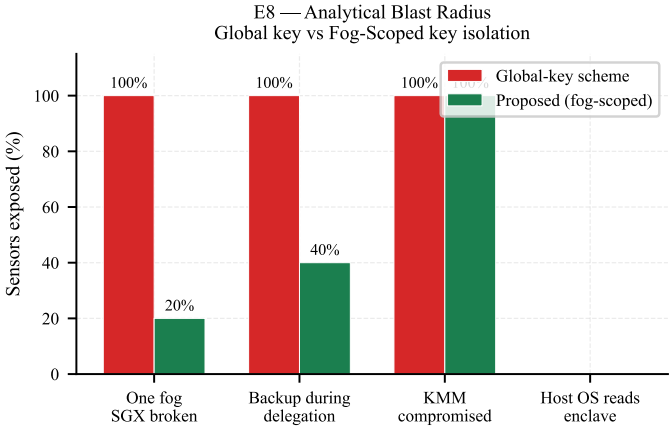

Fig. 11. Key exposure comparison between global and fog-scoped key designs across four threat scenarios (100 sensors, 20 per node).

VI. FUTURE WORK

A limitation of the present work is that the CapacityScore weights (w_1, w_2, w_3, w_4) (Eq. (34)) and the compatibility term coefficients are fixed heuristics selected by task classification rather than learned from observed workload distributions. The strategy comparison in E5 demonstrates the advantage of the compatibility-aware term over simpler scoring strategies, but the specific weight values motivate data-driven adaptation. An important direction for future work is to replace the static weight table with an online learning mechanism—such as a contextual bandit or lightweight reinforcement learning policy—that adjusts weights based on observed deadline

miss rates and node utilization histories across aggregation windows. Additionally, the workload update model in E5 uses a formula-based service time derived from static node profiles rather than a principled queuing model; extending the simulation to a discrete-event framework with explicit queue dynamics (e.g., M/M/c per node) would produce more accurate estimates of redelegation rates and deadline miss probabilities at larger scales. The present evaluation covers five fog nodes, 20 windows, and 46–58 tasks per window depending on the stress phase; characterizing system behavior under hundreds of nodes and high-variance arrival rates represents a necessary step toward deployment-scale validation. Furthermore, the current Paillier-based aggregation layer supports only additive operations—sum, mean, and weighted sum. Future work will extend the scheme to CKKS-based slot batching to support multiplicative aggregation functions such as variance, product aggregation, and nonlinear statistics, and to provide approximate arithmetic over packed sensor vectors with formally bounded noise growth.

The experimental evaluation rests on several simplifying assumptions that limit direct correspondence to real deployments. As described in Section V-A, the TEE layer is emulated using OP-TEE on QEMU; no physical enclave boundary, sealing-key derivation, or remote attestation exchange is present in the prototype. The KMM provisioning latency $T_{KMM} = 50\,\text{ms}$ is a fixed design parameter rather than a measured quantity. Future work should deploy the enclave layer on physical hardware—either an Intel SGX-capable processor using the Intel SGX SDK or Gramine, or an ARM TrustZone board running OP-TEE—and measure provisioning latency directly. Similarly, the end-to-end pipeline latency in E7 is derived entirely from hardcoded hardware-target constants; these estimates should be replaced by measurements on physical fog hardware equipped with AES-NI and a hardware TEE.

The fault tolerance evaluation in E6 is based on an analytical reading-loss model that counts sensor readings falling within a fixed failure-to-recovery time window under three predetermined failure timings. This model does not capture cascading failures, partial node degradation, network partition effects, or the interaction between simultaneous sensor ACK timeouts and the gossip convergence process. Future work should validate the proposed ACK+KMM recovery mechanism under realistic fault injection using network emulation tools such as tc-netem or a containerized testbed, and should extend the failure model to cover correlated multi-node failures and transient network outages. The gossip protocol parameters—fan-out k=3, miss threshold r=3, and gossip interval T_g=500 ms—are fixed throughout the evaluation; the probabilistic detection model derived from the Poisson failure process provides a design criterion ($T_g \leq \ln(20)/(\mu r)$ for $\beta = 0.95$), but adaptive parameter selection under varying failure rates μ has not been investigated. The KMM is currently modeled as a single trusted authority; future work may investigate threshold or multi-party key management constructions that distribute the trust anchor across a quorum of KMM instances, reducing the exposure when the KMM is compromised. Finally, the present scheme provides computational confidentiality under the IND-CPA assumption for Paillier and AES-GCM but does not address what an honest-but-curious cloud observer can infer from the sequence of decrypted aggregates over time. Investigating differential privacy mechanisms for the aggregate output, or bounding the statistical leakage of the published aggregate stream, remains an open problem for future work.

VII. Conclusion

This paper presented a fault-tolerant and load-balanced IIoT edge–fog architecture that jointly addresses three critical challenges: resource imbalance across heterogeneous fog nodes, fog-layer failures, and plaintext exposure during aggregation. The proposed framework integrates fog-scoped AES key isolation enforced by Trusted Execution Environments (TEEs), Paillier homomorphic slot aggregation with a zero-fill protocol for mid-window delegation, a three-layer fault-handling hierarchy combining per-reading TEE delegation, sensor ACK-timeout redirect, and gossip-based failure detection, and a task-aware CapacityScore selector that adapts routing decisions to the computational profile of each task type.

Experimental evaluation across eight experiments demonstrated: constant per-window cloud storage independent of sensor count (E1); aggregate correctness under mid-window delegation in all 100 trials for both single-source and multi-source delegation (E3a–E3b); KMM combine latency scaling as $\approx 1.474(k-1)$ ms, remaining within the 500 ms window for all tested $k \leq 100$ (E4); the highest deadline satisfaction rate of 78.77% and the lowest bandwidth-utilization variance among six compared strategies (E5); worst-case data loss reduced from 75% under gossip-only detection to 7.5% with the ACK+KMM mechanism at only $1.055\times$ message overhead (E6); and 60–80% reduction in key-exposure blast radius relative to a global-key design under single-node compromise (E8).

Future work will focus on validating the TEE enclave implementation on physical hardware (Intel SGX or ARM TrustZone), replacing the static CapacityScore weight table with an adaptive online learning policy, and extending the Paillier aggregation layer to CKKS-based slot batching for multiplicative and nonlinear aggregation functions.

References

[1] K. Tange, M. De Donno, X. Fafoutis, and N. Dragoni, "A systematic survey of industrial Internet of Things security: Requirements and fog computing opportunities," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 4, pp. 2489–2520, 2020.

[2] M. S. Farooq, M. Abdullah, S. Riaz, A. Alvi, F. Rustam *et al.*, "A survey on industrial Internet of Things security: Requirements, attacks, AI-based solutions, and edge computing opportunities," *Sensors*, vol. 23, no. 17, p. 7470, 2023.

[3] J. Jin, K. Yu, J. Kua, N. Zhang, Z. Pang, and Q.-L. Han, "Cloud-fog automation: Vision, enabling technologies, and future research directions," *IEEE Transactions on Industrial Informatics*, vol. 20, no. 2, pp. 1039–1054, 2024.

[4] M. H. Kashani and E. Mahdipour, "Load balancing algorithms in fog computing," *IEEE Transactions on Services Computing*, vol. 16, no. 2, pp. 1505–1521, 2023.

[5] S. Chen *et al.*, "A hybrid approach for fault-tolerance aware load balancing in fog computing," *Cluster Computing*, vol. 27, pp. 1–20, 2024.

[6] A. A. Mirani, G. Velasco-Hernandez, A. Awasthi, and J. Walsh, "Key challenges and emerging technologies in industrial IoT architectures: A review," *Sensors*, vol. 22, no. 15, p. 5836, 2022.

[7] C. Peng, M. Luo, P. Vijayakumar, D. He, O. Said, and A. Tolba, "Multifunctional and multidimensional secure data aggregation scheme in WSNs," *IEEE Internet of Things Journal*, vol. 9, no. 4, pp. 2657–2668, 2022, top-tier: IEEE Internet of Things Journal.

[8] V. Costan and S. Devadas, "Intel SGX explained," IACR Cryptology ePrint Archive, Tech. Rep. 2016/086, 2016. [Online]. Available: https://eprint.iacr.org/2016/086

[9] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Advances in Cryptology – EUROCRYPT 1999*, ser. Lecture Notes in Computer Science, vol. 1592. Springer, 1999, pp. 223–238.

[10] M. H. Kashani, A. Ahmadzadeh, and E. Mahdipour, "Load balancing mechanisms in fog computing: A systematic review," 2020, arXiv:2011.14706.

[11] M. Vijarania *et al.*, "A systematic literature review on load-balancing techniques in fog computing: Architectures, strategies, and emerging trends," *Computers*, vol. 14, no. 6, p. 217, 2025.

[12] A. Nemati and N. Mansouri, "Resource allocation in fog computing: A survey on current state and research challenges," *Knowledge and Information Systems*, 2024.

[13] N. Tripathy *et al.*, "Energy and makespan optimised task mapping in fog enabled IoT application: A hybrid approach," *Scientific Reports*, 2026.

[14] M. Kaur and R. Aron, "FOCALB: Fog computing architecture of load balancing for scientific workflow applications," *Journal of Grid Computing*, vol. 19, no. 4, p. 40, 2021.

[15] M. Ebrahim and A. Hafid, "Resilience and load balancing in fog networks: A multi-criteria decision analysis approach," *Microprocessors and Microsystems*, vol. 101, p. 104893, 2023.

[16] A. Ahmed, A. Alsharkawy, M. E. Embabi, and A. Emara, "Adaptive multi-criteria-based load balancing technique for resource allocation in fog-cloud environments," *International Journal of Computer Networks and Communications*, vol. 16, no. 1, pp. 105–124, 2024.

[17] A. Al-Shafei, H. Zareipour, and Y. Cao, "Cited in IEEE COMST survey on AI-empowered fog/edge resource management," *IEEE Communications Surveys & Tutorials*, 2023.

[18] M. Ebrahim and A. Hafid, "Privacy-aware load balancing in fog networks: A reinforcement learning approach," 2023, arXiv:2301.09497.

[19] M. Ebrahim, "Fully distributed fog load balancing with multi-agent reinforcement learning," 2025, arXiv:2405.12236.

[20] M. Kirti, A. K. Maurya, and R. S. Yadav, "Fault-tolerance approaches for distributed and cloud computing environments: A systematic review, taxonomy and future directions," *Concurrency and Computation: Practice and Experience*, vol. 36, no. 13, p. e8081, 2024.

[21] Z. Liu *et al.*, "A fault-tolerant scheduling algorithm that minimises the number of replicas in heterogeneous service-oriented cloud computing systems," *The Journal of Supercomputing*, 2024.

[22] H. T. Rajab and M. F. Younis, "Dynamic fault tolerance aware scheduling for healthcare system on fog computing," *Iraqi Journal of Science*, vol. 62, no. 1, pp. 308–318, 2021.

[23] A. Ghasemi, "MOHHO: Multi-objective Harris Hawks optimization algorithm for service placement in fog computing," *The Journal of Supercomputing*, 2024.

[24] M. Merah, Z. Aliouat, and H. Mabed, "Dynamic load balancing of traffic in the IoT edge computing environment using a clustering approach based on deep learning and genetic algorithms," *Cluster Computing*, 2024.

[25] V. Mohammadi, A. M. Rahmani, A. Darwesh, and A. Sahafi, "Fault tolerance in fog-based Social Internet of Things," *Knowledge-Based Systems*, vol. 265, p. 110376, 2023.

[26] A. Demers, D. Greene, C. Houser, W. Irish, J. Larson, S. Shenker, H. Sturgis, D. Swinehart, and D. Terry, "Epidemic algorithms for replicated database maintenance," *ACM SIGOPS Operating Systems Review*, vol. 22, no. 1, pp. 8–32, 1988.

[27] R. Naha *et al.*, "Fuzzy logic-based robust failure handling mechanism for fog computing," 2021, arXiv:2103.06381.

[28] Wang *et al.*, "Privacy-preserving data aggregation against false data injection attacks in fog computing," 2022, pMC6111540.

[29] X. Shang *et al.*, "Fault-tolerant data aggregation combining Paillier homomorphic encryption and ECDSA signatures," 2023, cited in Journal of King Saud University survey on privacy-preserving aggregation, 2025, doi:10.1007/s44443-025-00179-z.

[30] "IoT-enabled fog-based secure aggregation in smart grids supporting data analytics (FESDAO)," 2025, pMC12526643.

[31] A. Januszewicz *et al.*, "MOZAIK: A privacy-preserving analytics platform for IoT data using MPC and FHE," *Journal of Network and Systems Management*, 2025.

[32] Z. Qin, H. Xiong, S. Wu, and J. Batamuliza, "A survey of proxy re-encryption for cloud data sharing," *IEEE Transactions on Services Computing*, vol. 15, no. 4, pp. 1964–1982, 2022.

[33] M. Kumar *et al.*, "AI-based sustainable and intelligent offloading framework for IIoT in collaborative cloud-fog environments," *IEEE Transactions on Consumer Electronics*, 2023.

[34] M. Abdel-Basset, G. Manogaran, M. Mohamed, and E. Rushdy, "A fault-tolerant aware scheduling method for fog-cloud environments," *Future Generation Computer Systems*, vol. 93, pp. 1–12, 2019.

[35] R. Liu, W. Wu, X. Guo, G. Zeng, and K. Li, "Replica fault-tolerant scheduling with time guarantee under energy constraint in fog computing," *Future Generation Computer Systems*, vol. 159, pp. 567–579, 2024.

[36] M. H. Shahab, Y. Sharma, A. Jindal, and A. Al-Dulaimy, "A bi-objective policy for resilient and sustainable sfc management in telco-cloud environments," *IEEE Access*, vol. 13, 2025.

[37] Y.-L. Lee, D. Liang, and W.-J. Wang, "Optimal online liveness fault detection for multilayer cloud computing systems," *IEEE Transactions on Dependable and Secure Computing*, vol. 19, no. 5, pp. 3464–3479, 2022.

[38] V. Mohammadi, A. M. Rahmani, A. Darwesh, and A. Sahafi, "Fault tolerance in fog-based social internet of things," *Knowledge-Based Systems*, vol. 265, p. 110376, 2023.